

32-AI-QUALITY-STANDARDS.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Quality Framework

Projet : Vitala

1. Objectif

AI-QUALITY-STANDARDS.md définit les standards minimums de qualité applicables à tous les composants de Vitala.

Ces standards concernent :

- architecture ;
 - code ;
 - sécurité ;
 - expérience utilisateur ;
 - performance ;
 - données ;
 - documentation.
-

2. Principe fondamental

Une fonctionnalité Vitala doit être :

``text id="k9m3vq" Fonctionnelle

+

Sécurisée

+

Maintenable

+

Évolutive

+

Compréhensible

Une fonctionnalité qui remplit seulement un de ces critères est incomplète.

3. Standard Architecture

Toute fonctionnalité doit respecter :

Modularité

Le code doit être organisé par domaine.

Exemple :

```
```text id="9qz2ap"
features/

 flash/

 scan/

 radar/

 veille/

 admin/
```

---

## Séparation des responsabilités

Séparer :

- interface ;
- logique métier ;
- accès données ;
- services ;
- types.

---

## Faible couplage

Un module ne doit pas dépendre inutilement d'un autre module.

---

## 4. Standard Code

Le code Vitala doit respecter :

### Lisibilité

Le code doit être compréhensible par :

- une autre IA ;
  - un développeur ;
  - un futur collaborateur.
- 

### Simplicité

Préférer :

- solutions simples ;
- fonctions courtes ;
- logique claire.

Éviter :

- complexité inutile ;
  - abstractions excessives.
- 

### Cohérence

Respecter :

- conventions de nommage ;
  - structure projet ;
  - patterns existants.
- 

## 5. Standard TypeScript

Le code TypeScript doit favoriser :

- typage strict ;
- interfaces claires ;
- absence de `any` inutile ;
- validation des données.

Interdit :

- ignorer les erreurs de type volontairement.
- 

## 6. Standard Sécurité

Toute fonctionnalité doit intégrer :

### Authentification

Vérifier l'identité.

---

### Autorisation

Vérifier les permissions.

---

### Données

Protéger les informations sensibles.

---

### API

Valider toutes les entrées.

---

## 7. Standard Database

La base de données doit respecter :

- schéma documenté ;
  - migrations propres ;
  - relations cohérentes ;
  - index adaptés ;
  - RLS activé.
- 

## 8. Standard API

Toute API doit fournir :

- contrat clair ;

- réponses cohérentes ;
  - erreurs structurées ;
  - sécurité ;
  - documentation.
- 

## 9. Standard Frontend

L'expérience utilisateur doit respecter :

### Interface

- claire ;
  - moderne ;
  - cohérente ;
  - responsive.
- 

### États obligatoires

Chaque écran doit gérer :

``text id="5n8xqp" Chargement

Succès

Erreur

Vide

Permission refusée ``

---

## 10. Standard UX

Une fonctionnalité doit être :

- intuitive ;
- rapide à comprendre ;
- accessible.

L'utilisateur ne doit pas avoir besoin d'explication technique.

---

## 11. Standard Performance

L'IA doit rechercher :

- temps de chargement excessif ;
- requêtes inutiles ;
- traitements coûteux.

Objectif :

Une expérience fluide même avec une montée en charge.

---

## 12. Standard Offline First

Pour les modules concernés :

Le système doit prévoir :

- fonctionnement hors connexion ;
  - stockage local ;
  - synchronisation ;
  - résolution conflits.
- 

## 13. Standard Intelligence Artificielle

Toute fonctionnalité IA doit respecter :

- contrôle des coûts ;
  - protection des données ;
  - limites claires ;
  - gestion des erreurs.
- 

## 14. Standard Tests

Une fonctionnalité doit avoir :

- tests adaptés ;
  - validation des cas normaux ;
  - validation des erreurs.
-

## 15. Standard Documentation

Chaque élément important doit être documenté :

- architecture ;
  - décisions ;
  - utilisation ;
  - contraintes.
- 

## 16. Standard Scalabilité

Une solution doit être pensée pour évoluer.

Éviter :

- limites artificielles ;
  - architecture bloquante ;
  - dépendances inutiles.
- 

## 17. Niveau de qualité par criticité

### Fonction critique

Exigence maximale :

- tests complets ;
  - sécurité renforcée ;
  - documentation complète.
- 

### Fonction standard

Exigence normale :

- tests principaux ;
  - documentation adaptée.
- 

### Fonction expérimentale

Peut évoluer rapidement mais doit être isolée.

---

## 18. Critères "Production Ready"

Une fonctionnalité est prête si :

☐ Fonctionne correctement

☐ Sécurité validée

☐ Tests validés

☐ Documentation présente

☐ Performance acceptable

☐ Architecture respectée

---

## 19. Interdictions

Une IA ne doit jamais livrer :

✗ code non testé.

✗ fonctionnalité non sécurisée.

✗ architecture improvisée.

✗ duplication inutile.

✗ solution temporaire présentée comme définitive.

---

## 20. Règle finale

La qualité Vitala n'est pas basée sur la quantité de code produit.

Elle est basée sur :

- la fiabilité ;
  - la simplicité ;
  - la sécurité ;
  - la capacité d'évolution.
-



# Historique

Version	Modification
V1.0	Création des standards qualité IA

---

**Fin de AI/32-AI-QUALITY-STANDARDS.md**